

ANALYZING MALICIOUS DOCUMENTS

This cheat sheet outlines tips and tools for analyzing malicious documents, such as Microsoft Office, RTF, and PDF files.

General Approach to Document Analysis

1. Examine the document for anomalies, such as risky tags, scripts, and embedded artifacts.
2. Locate embedded code, such as shellcode, macros, JavaScript, or other suspicious objects.
3. Extract suspicious code or objects from the file.
4. If relevant, deobfuscate and examine macros, JavaScript, or other embedded code.
5. If relevant, emulate, disassemble and/or debug shellcode that you extracted from the document.
6. Understand the next steps in the infection chain.

Microsoft Office Format Notes

Binary Microsoft Office document files (.doc, .xls, etc.) use the OLE2 (a.k.a. Structured Storage) format.

SRP streams in OLE2 documents sometimes store a cached version of earlier VBA macro code.

OOXML document files (.docx, .xlsm, etc.) supported by Microsoft Office are compressed zip archives.

VBA macros in OOXML documents are stored inside an OLE2 binary file, which is within the zip archive.

Excel supports XLM macros that are embedded as formulas in sheets without the OLE2 binary file.

RTF documents don't support macros but can contain malicious embedded files and objects.

Useful MS Office File Analysis Commands

<code>zipdump.py file.pptx</code>	Examine contents of OOXML file <i>file.pptx</i> .
<code>zipdump.py file.pptx -s 3 -d</code>	Extract file with index 3 from <i>file.pptx</i> to STDOUT.
<code>olevba file.xlsm</code>	Locate and extract macros from <i>file.xlsm</i> .

<code>oledump.py file.xls -i</code>	List all OLE2 streams present in <i>file.xls</i> .
<code>oledump.py file.xls -s 3 -v</code>	Extract VBA source code from stream 3 in <i>file.xls</i> .
<code>xmldump.py pretty</code>	Format XML file supplied via STDIN for easier analysis.
<code>oledump.py file.xls -p plugin_http_heuristics</code>	Find obfuscated URLs in <i>file.xls</i> macros.
<code>vmonkey file.doc</code>	Emulate the execution of macros in <i>file.doc</i> to analyze them.
<code>evilclippy -uu file.ppt</code>	Remove the password prompt from macros in <i>file.ppt</i> .
<code>msoffcrypto-tool infile.docm outfile.docm -p</code>	Decrypt <i>outfile.docm</i> using specified password to create <i>outfile.docm</i> .
<code>pcodeomp file.doc</code>	Disassemble VBA-stomped p-code macro from <i>file.doc</i> .
<code>pcode2code file.doc</code>	Decompile VBA-stomped p-code macro from <i>file.doc</i> .
<code>rtfobj.py file.rtf</code>	Extract objects embedded into RTF <i>file.rtf</i> .
<code>rtfdump.py file.rtf</code>	List groups and structure of RTF file <i>file.rtf</i> .
<code>rtfdump.py file.rtf -0</code>	Examine objects in RTF file <i>file.rtf</i> .
<code>rtfdump.py file.rtf -s 5 -H -d</code>	Extract hex contents from group in RTF file <i>file.rtf</i> .
<code>xlmdeobfuscator --file file.xlsm</code>	Deobfuscate XLM (Excel 4) macros in <i>file.xlsm</i> .

Risky PDF Keywords

/OpenAction and /AA specify the script or action to run automatically.

/JavaScript, /JS, /AcroForm, and /XFA can specify JavaScript to run.

/URI accesses a URL, perhaps for phishing.

/SubmitForm and /GoToR can send data to URL.

/ObjStm can hide objects inside an object stream.

/XObject can embed an image for phishing.

Be mindful of obfuscation with hex codes, such as /JavaScript vs. /J#61vaScript. (See examples.)

Useful PDF File Analysis Commands

<code>pdfid.py file.pdf -n</code>	Display risky keywords present in file <i>file.pdf</i> .
<code>pdf-parser.py file.pdf -a</code>	Show stats about keywords. Add "-O" to include object streams.
<code>pdf-parser.py file.pdf -o id</code>	Display contents of object <i>id</i> . Add "-d" to dump object's stream.
<code>pdf-parser.py file.pdf -r id</code>	Display objects that reference object <i>id</i> .
<code>qpdf --password=pass --decrypt infile.pdf outfile.pdf</code>	Decrypt <i>infile.pdf</i> using password <i>pass</i> to create <i>outfile.pdf</i> .

Shellcode and Other Analysis Commands

<code>xorsearch -W -d 3 file.bin</code>	Locate shellcode patterns inside the binary file <i>file.bin</i> .
<code>sdcdbg /f file.bin</code>	Emulate execution of shellcode in <i>file.bin</i> . Use "/off" to specify offset.
<code>runsc32 -f file.bin -n</code>	Execute shellcode in <i>file.bin</i> to observe behavior in an isolated lab.
<code>base64dump.py file.txt</code>	List Base64-encoded strings present in file <i>file.txt</i> .
<code>numbers-to-string.py file</code>	Convert numbers that represent characters in <i>file</i> to a string.

Additional Document Analysis Tools

SpiderMonkey, cscript, and box-js help deobfuscate JavaScript that you extract from document files.

Use the debugger built into Microsoft Office to deobfuscate macros in an isolated lab.

Use AMSIScriptContentRetrieval.ps1 to observe Microsoft Office execute macros in an isolated lab.

Some automated analysis sandboxes can analyze aspects of malicious document files.

REMnux distro includes many of the free document analysis tools mentioned above.